

Modul Praktikum Flutter Lanjutan

Pertemuan 13: Upload Foto Bukti Struk dan Pengujian Otomatis

Durasi	Level	Bentuk kerja	Prasyarat
150 menit	Lanjut	Individu	Pertemuan 12 (stack lengkap: API, admin, aplikasi)

1. Tujuan Pembelajaran

Setelah praktikum ini, mahasiswa mampu:

1. Menerima unggahan berkas di FastAPI (`multipart/form-data`) dengan validasi jenis, isi, dan ukuran.
2. Menyimpan berkas di *volume* Docker dan menyajikannya sebagai berkas statis.
3. Mengambil foto dari kamera/galeri di Flutter (`image_picker`) lalu mengunggahnya dengan `http.MultipartRequest`.
4. Menampilkan berkas yang sama di aplikasi dan dashboard Filament.
5. Menulis pengujian otomatis backend dengan **pytest** dan database uji terpisah.

2. Alat dan Bahan

- Hasil pertemuan 12 (atau snapshot `pert12/tugas12`).
- Paket Flutter baru: `image_picker`, `http_parser`.
- Paket Python untuk pengujian: `pytest`, `httpx2` (dipakai `TestClient`).
- Gambar contoh (JPG/PNG) di emulator. Kamera emulator Android dapat memakai *virtual scene*.

3. Teori Singkat

Multipart. JSON tidak cocok untuk mengirim berkas biner. Formulir `multipart/form-data` membagi badan request menjadi beberapa bagian; setiap bagian memiliki nama *field*, nama berkas, dan `Content-Type`:

```
POST /kegiatan/5/bukti
Content-Type: multipart/form-data; boundary=----abc

-----abc
Content-Disposition: form-data; name="berkas"; filename="struk.jpg"
Content-Type: image/jpeg

<byte gambar>
-----abc--
```

Jangan percaya klien. `Content-Type` dan nama berkas dikirim oleh klien dan bisa dipalsukan. Server harus: (1) membatasi ukuran, (2) memeriksa *isi* berkas lewat *magic bytes* (JPEG diawali `FF D8 FF`, PNG diawali `89 50 4E 47 0D 0A 1A 0A`), dan (3) membuat nama berkas sendiri (UUID acak). Dengan begitu, nama seperti `../.. /app/main.py` tidak berbahaya dan URL foto orang lain tidak dapat ditebak.

Di mana berkas disimpan? Di database hanya disimpan **path relatif** (`uploads/3f2a....jpg`). Berkasnya sendiri ada di *volume* Docker `uploads`, sehingga tidak hilang saat container dibuat ulang. Klien menyusun URL lengkap: `alamatApi + "/" + bukti`. Pendekatan ini penting karena alamat API berbeda di setiap perangkat (10.0.2.2, IP laptop, atau domain produksi).

Piramida pengujian.

Jenis	Contoh di TabungKu	Kecepatan
Unit test	<code>jenisGambar('a.png'), Kegiatan.fromJson</code>	Sangat cepat
Integrasi/API test	<code>pytest + TestClient + SQLite memori</code>	Cepat (detik)
Widget test	BerandaPage dengan FakeRepository	Cepat
End-to-end	Aplikasi di emulator + Docker sungguhan	Lambat; dibahas di pertemuan 14

4. Langkah Praktikum

Bagian A: Kolom dan Penyimpanan Bukti

1. Tambahkan ke Settings:

```
# Folder foto bukti struk (di docker-compose dipasang sebagai volume).
upload_dir: str = "/data/uploads"
maks_ukuran_bukti: int = 2 * 1024 * 1024 # 2 MB
```

2. Tambahkan kolom ke model Kegiatan, lalu buat revisi `0004_bukti_kegiatan.py` berisi `op.add_column("kegiatan", sa.Column("bukti", sa.String(255), nullable=True))`:

```
# Path relatif foto bukti, misalnya "uploads/3f2a...jpg".
bukti: Mapped[str | None] = mapped_column(String(255))
```

Tambahkan juga bukti: `str | None = None` ke `KegiatanOut`.

3. Pasang volume di layanan api (`docker-compose.yml`) dan daftarkan `uploads:` di bagian `volumes:` paling bawah:

```
volumes:
  - ./backend:/code
  - uploads:/data/uploads # foto bukti struk tetap ada walau container dibuat ulang
```

4. Sajikan folder sebagai berkas statis di `main.py`:

```
# Foto bukti dapat dibuka langsung: GET /uploads/<nama-berkas>.
# Nama berkas acak (UUID) sehingga tidak dapat ditebak.
os.makedirs(settings.upload_dir, exist_ok=True)
app.mount("/uploads", StaticFiles(directory=settings.upload_dir), name="uploads")
```

Bagian B: Validasi dan Endpoint Unggah

`app/bukti.py`:

```
# Tanda tangan (magic bytes) di awal berkas. Content-Type dari klien
# bisa dipalsukan, jadi isi berkas juga diperiksa.
_FORMAT = {
    "image/jpeg": (b"\xff\xd8\xff", ".jpg"),
    "image/png": (b"\x89PNG\r\n\x1a\n", ".png"),
    "image/webp": (b"RIFF", ".webp"),
}

async def simpan_bukti(berkas: UploadFile) -> str:
    """Validasi lalu simpan berkas. Mengembalikan path relatif 'uploads/xxx.jpg'."""
    if berkas.content_type not in _FORMAT:
        raise HTTPException(
            status.HTTP_415_UNSUPPORTED_MEDIA_TYPE, "Format harus JPG, PNG, atau WEBP"
        )
    # Baca maksimal 1 byte melebihi batas untuk mengetahui berkas terlalu besar.
    isi = await berkas.read(settings.maks_ukuran_bukti + 1)
    if len(isi) > settings.maks_ukuran_bukti:
```

```

        raise HTTPException(status.HTTP_413_CONTENT_TOO_LARGE, "Ukuran foto maksimal 2 MB")

tanda, ekstensi = _FORMAT[berkas.content_type]
if not isi.startswith(tanda):
    raise HTTPException(
        status.HTTP_415_UNSUPPORTED_MEDIA_TYPE, "Isi berkas bukan gambar yang valid"
    )

nama = uuid.uuid4().hex + ekstensi
with open(os.path.join(settings.upload_dir, nama), "wb") as f:
    f.write(isi)
return f"uploads/{nama}"

def hapus_bukti(path_relatif: str | None) -> None:
    if not path_relatif:
        return
    path = os.path.join(settings.upload_dir, os.path.basename(path_relatif))
    if os.path.exists(path):
        os.remove(path)

```

Endpoint di `routers/kegiatan.py` (`UploadFile` membutuhkan `python-multipart`, yang sudah terpasang sejak pertemuan 10):

```

@router.post("/{kegiatan_id}/bukti", response_model=KegiatanOut)
async def unggah_bukti(kegiatan_id: int, berkas: UploadFile, db: DbSession, user: UserAktif):
    """Unggah foto struk (multipart/form-data, field `berkas`). Foto lama diganti."""
    k = _ambil_atau_404(db, kegiatan_id, user)
    if k.tipe == Tipe.tanpa:
        raise HTTPException(
            status.HTTP_400_BAD_REQUEST, "Bukti hanya untuk pemasukan/pengeluaran"
        )
    lama = k.bukti
    k.bukti = await simpan_bukti(berkas)
    db.commit()
    db.refresh(k)
    hapus_bukti(lama)
    return k

```

Buat juga `DELETE /kegiatan/{id}/bukti`. Di `hapus_kegiatan`, hapus berkasnya **setelah** commit berhasil agar tidak ada berkas yang terhapus untuk data yang ternyata gagal dihapus.

Checkpoint B: di Swagger, POST `/kegiatan/{id}/bukti` dengan foto mengembalikan "bukti": `"uploads/....jpg"`, dan `http://localhost:8000/uploads/....jpg` menampilkan fotonya. PDF dibalas 415, sedangkan berkas teks yang diberi nama `.jpg` juga dibalas 415.

Bagian C: Mengambil dan Mengunggah Foto di Flutter

1. Tambahkan `image_picker: ^1.2.4` dan `http_parser: ^4.1.2`. Untuk iOS, tambahkan ke `ios/Runner/Info.plist`:

```

<key>NSCameraUsageDescription</key>
<string>Memotret struk sebagai bukti pengeluaran.</string>
<key>NSPhotoLibraryUsageDescription</key>
<string>Memilih foto struk sebagai bukti pengeluaran.</string>

```

2. Model Kegiatan mendapat final `String? bukti`; (dibaca dari JSON) dan `getter` URL lengkap:

```

/// URL lengkap foto bukti. Disusun di klien karena alamat API dapat
/// berbeda (emulator 10.0.2.2, HP fisik memakai IP laptop).
String? get buktiUrl => bukti == null ? null : '${apiUrl.value}/${bukti}';

```

3. Unggah dengan `MultipartRequest` di `KegiatanApi`. Request dikirim lewat `_client` yang sama, sehingga header token dari `ApiClient` ikut terpasang:

```

Future<Kegiatan> unggahBukti(int id, List<int> isi, String namaBerkas) async {
  final req = http.MultipartRequest('POST', _uri('/kegiatan/$id/bukti'))
  ..files.add(
    http.MultipartFile.fromBytes(
      'berkas',
      isi,
      filename: namaBerkas,
      contentType: MediaType.parse(jenisGambar(namaBerkas)),
    ),
  );
  // Unggah butuh waktu lebih lama daripada request biasa.
  final res = _cek(
    await http.Response.fromStream(
      await _client.send(req).timeout(const Duration(seconds: 60)),
    ),
  );
  return Kegiatan.fromJson(jsonDecode(res.body) as Map<String, dynamic>);
}

```

`jenisGambar()` menentukan `image/jpeg`, `image/png`, atau `image/webp` dari ekstensi berkas. Tambahkan `unggahBukti` dan `hapusBukti` ke kontrak repository. Ubah juga tambah/ubah agar ****mengembalikan Kegiatan****, karena id kegiatan baru dibutuhkan untuk mengunggah foto. Di `SqliteKegiatanRepository`, kedua method bukti melempar `ApiException('Bukti struk hanya tersedia saat memakai server')`.

4. Widget `lib/widgets/bukti_struk.dart` berisi pratinjau foto, tombol **Kamera**, **Galeri**, **Hapus foto**, dan tampilan layar penuh dengan `InteractiveViewer` (cubit untuk memperbesar). Foto diperkecil di HP sebelum diunggah:

```

Future<void> _ambil(BuildContext context, ImageSource sumber) async {
  try {
    // Diperkecil di HP agar unggahan cepat dan di bawah batas 2 MB server.
    final f = await ImagePicker().pickImage(
      source: sumber,
      maxWidth: 1280,
      imageQuality: 80,
    );
    if (f == null) return; // pengguna membatalkan
    onPilih(FotoBaru(await f.readAsBytes(), f.name));
  } catch (e) {
    if (!context.mounted) return;
    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(content: Text('Tidak dapat membuka kamera/galeri')),
    );
  }
}

```

Foto baru ditampilkan dengan `Image.memory`, sedangkan foto yang sudah ada di server memakai `Image.network(urlLama)` dengan `loadingBuilder` dan `errorBuilder`.

5. Di form, bagian **Bukti struk (opsional)** muncul bila jenisnya bukan "Tidak". Foto diunggah **setelah** kegiatan tersimpan:

```

final Kegiatan tersimpan;
try {
  tersimpan = widget.kegiatan == null ? await repo.tambah(k) : await repo.ubah(k);
} catch (e) {
  if (mounted) {
    tampilkanGalat(context, e);
    setState(() => _menyimpan = false);
  }
  return;
}

// Foto diunggah SETELAH kegiatan tersimpan karena butuh id kegiatan.
try {
  final foto = _fotoBaru;

```

```

    if (adaUang && foto != null) {
        await repo.unggahBukti(tersimpan.id!, foto.isi, foto.nama);
    } else if ((_hapusFotoLama || !adaUang) && widget.kegiatan?.bukti != null) {
        await repo.hapusBukti(tersimpan.id!);
    }
} catch (e) {
    // Kegiatan sudah tersimpan; cukup beri tahu bahwa fotonya gagal.
    ...
}
if (mounted) Navigator.pop(context);

```

6. KegiatanTile menampilkan ikon Icons.receipt_long bila kegiatan memiliki bukti.

Checkpoint C: tambah pengeluaran beserta foto dari galeri. Ikon struk muncul di daftar, dan membuka form menampilkan fotonya. Ganti foto, lalu periksa di container bahwa berkas lama terhapus: `docker compose exec api ls /data/uploads`.

Bagian D: Foto di Dashboard Filament

Browser admin mengambil foto dari FastAPI, jadi Laravel cukup mengetahui alamat publik API.

Tambahkan ke `config/services.php`:

```

'tabungku_api' => [
    'url' => env('API_PUBLIC_URL', 'http://localhost:8000'),
],

```

Tambahkan `API_PUBLIC_URL: http://localhost:8000` pada *environment* layanan admin, lalu tambahkan method di model Kegiatan (Laravel):

```

/** URL lengkap foto bukti, disajikan oleh FastAPI di /uploads. */
public function buktiUrl(): ?string
{
    return $this->bukti
        ? rtrim(config('services.tabungku_api.url'), '/') . '/' . $this->bukti
        : null;
}

```

Di KegiatanResource, tambahkan kolom `ImageColumn::make('bukti')->state(fn (Kegiatan $record): ?string => $record->buktiUrl())->imageHeight(40)` dan `ImageEntry` serupa (tinggi 320) di *infolist* untuk modal **Lihat**.

Checkpoint D: *thumbnail* struk tampil di tabel Kegiatan, dan modal Lihat menampilkan foto ukuran besar.

Bagian E: Pengujian Backend dengan pytest

1. Buat `backend/requirements-dev.txt`, lalu ubah `Dockerfile` agar memasang berkas ini (COPY `requirements.txt requirements-dev.txt ./` lalu `pip install -r requirements-dev.txt`):

```

# Dependensi tambahan untuk pengujian (tidak dibutuhkan di produksi).
-r requirements.txt
pytest==9.1.1
httpx==2.13.1 # dipakai TestClient FastAPI

```

2. `tests/conftest.py` mengganti database dengan **SQLite di memori** lewat `dependency_overrides`. Pengujian menjadi cepat dan tidak menyentuh data di MySQL:

```

import os
import tempfile

# Harus di-set sebelum modul app diimpor (Settings dibaca saat impor).
os.environ["DATABASE_URL"] = "sqlite://"

```

```

os.environ["UPLOAD_DIR"] = tempfile.mkdtemp(prefix="tabungku-uji-")

import pytest # noqa: E402
from fastapi.testclient import TestClient # noqa: E402
from sqlalchemy import create_engine # noqa: E402
from sqlalchemy.orm import sessionmaker # noqa: E402
from sqlalchemy.pool import StaticPool # noqa: E402

from app.database import Base, get_db # noqa: E402
from app.main import app # noqa: E402
from app.models import Kategori # noqa: E402

KATEGORI = ["Makan", "Transport", "Belajar", "Hiburan", "Lainnya"]

@pytest.fixture
def db_session():
    # StaticPool: semua koneksi memakai database memori yang sama.
    engine = create_engine(
        "sqlite://", connect_args={"check_same_thread": False}, poolclass=StaticPool
    )
    Base.metadata.create_all(engine)
    Sesi = sessionmaker(bind=engine, autoflush=False, expire_on_commit=False)
    with Sesi() as db:
        db.add_all(Kategori(nama=n, urutan=i) for i, n in enumerate(KATEGORI, 1))
        db.commit()
    yield Sesi
    engine.dispose()

@pytest.fixture
def client(db_session):
    def get_db_uji():
        db = db_session()
        try:
            yield db
        finally:
            db.close()

    # Ganti dependency get_db dengan database uji.
    app.dependency_overrides[get_db] = get_db_uji
    with TestClient(app) as c:
        yield c
    app.dependency_overrides.clear()

def daftar_dan_login(client: TestClient, email: str = "budi@contoh.id") -> dict:
    """Mendaftarkan akun lalu mengembalikan header Authorization."""
    client.post(
        "/auth/register", json={"nama": "Budi", "email": email, "password": "rahasial23"}
    )
    res = client.post("/auth/login", data={"username": email, "password": "rahasial23"})
    return {"Authorization": f"Bearer {res.json()['access_token']}"}

@pytest.fixture
def auth(client) -> dict:
    return daftar_dan_login(client)

```

SQLite untuk uji, MySQL untuk produksi. Cara ini cepat, tetapi beberapa perilaku khusus MySQL (misalnya UPDATE ... JOIN pada migrasi 0003) tidak teruji. Migrasi diuji terpisah dengan alembic downgrade/upgrade di MySQL (pertemuan 12).

3. Contoh test, tests/test_laporan.py, menguji skenario modul 7 lewat HTTP:

```

def test_skenario_satu_hari_modul_7(client, auth):
    data = [
        ("Kuliah pagi dan makan siang", "pengeluaran", 15000, 1, True),
        ("Pulang naik angkot", "pengeluaran", 10000, 2, True),
        ("Kerja freelance desain poster", "pemasukan", 50000, None, False),
    ]

```

```

        ("Belajar Flutter di perpustakaan", "tanpa", 0, None, False),
    ]
    for judul, tipe, nominal, kategori_id, selesai in data:
        res = client.post("/kegiatan", headers=auth, json={
            "judul": judul, "tanggal": "2026-10-05", "tipe": tipe,
            "nominal": nominal, "kategori_id": kategori_id, "selesai": selesai,
        })
        assert res.status_code == 201, res.text

    lap = client.get("/laporan/harian", headers=auth, params={"tanggal": "2026-10-05"}).json()
    assert lap["jumlah_kegiatan"] == 4
    assert lap["selesai"] == 2
    assert lap["tabungan"] == 25000
    assert lap["per_kategori"] == [
        {"kategori": "Makan", "total": 15000},
        {"kategori": "Transport", "total": 10000},
    ]

```

4. Lengkapi test untuk: autentikasi (`test_auth.py`: register, email ganda, password pendek, login salah, tanpa token, refresh token, akun nonaktif, ubah target), kegiatan (`test_kegiatan.py`: CRUD, aturan bisnis, PATCH tetap tervalidasi, **isolasi antar-pengguna**, kategori nonaktif), dan bukti (`test_bukti.py`: unggah lalu unduh, ganti foto menghapus berkas lama, 415/413, bukti untuk jenis tanpa uang ditolak, hapus bukti/kegiatan menghapus berkas). Contoh unggah di test:

```

JPG = b"\xff\xd8\xff\xe0" + b"0" * 100

def _unggah(client, auth, kid, isi=JPG, jenis="image/jpeg"):
    return client.post(
        f"/kegiatan/{kid}/bukti", headers=auth, files={"berkas": (isi, jenis)}
    )

```

5. Jalankan:

```

docker compose up -d --build api
docker compose exec api pytest -q

```

Checkpoint E: seluruh test lulus (snapshot: 22 test dalam sekitar 3 detik). Coba rusak satu aturan bisnis secara sengaja, misalnya hapus pengecekan `nominal <= 0`, lalu pastikan ada test yang gagal.

5. Latihan Mandiri

1. Tambahkan test Flutter untuk `KegiatanApi.unggahBukti` yang memeriksa header `multipart/form-data` dan nama field berkas (gunakan `MockClient`).
2. Buat *fixture* `kegiatan_contoh` yang membuat satu pengeluaran, lalu pakai untuk menyederhanakan `test_bukti.py`.
3. Pada aplikasi, tampilkan ukuran berkas (KB) di bawah pratinjau sebelum diunggah.
4. Diskusikan risiko keamanan folder `/uploads` yang bersifat publik. Bagaimana cara menyajikan foto **hanya** kepada pemiliknya?

6. Tugas

- Backend: GET `/laporan/bulanan?tahun=&bulan=` membalas total sebulan (kegiatan, selesai, pemasukan, pengeluaran, tabungan), pengeluaran per kategori, `jumlah_hari`, dan `hari_target_tercapai` (hari yang sudah lewat dengan tabungan $\geq$ target pengguna). Validasi bulan 1–12.
- Backend: GET `/laporan/ekspor.csv?mulai=&sampai=` mengunduh kegiatan pengguna sebagai CSV (`tanggal, judul, tipe, kategori, nominal, selesai`) dengan header

`Content-Disposition: attachment`. Rentang maksimal 366 hari.

- Test pytest untuk kedua endpoint, lalu ukur **cakupan kode** dengan `pytest-cov` (`pytest --cov --cov-report=term-missing`). Target cakupan minimal 85%.
- Filament: tombol **Ekspor CSV** di halaman Kegiatan yang mengekspor baris **sesuai filter dan pencarian yang sedang aktif** (`getFilteredSortedTableQuery()` + `response()->streamDownload`), beserta **feature test** `assertFileDownloaded()`.
- Flutter: kartu **Bulan ini** di halaman Laporan (total, tabungan, dan "Target harian tercapai X dari Y hari" dengan *progress bar*), beserta test.
- Kumpulkan: kode, hasil `pytest --cov`, berkas CSV hasil ekspor (dari API dan dari dashboard), dan tangkapan layar kartu bulanan.

7. Rubrik Penilaian

Komponen	Bobot
Upload aman di backend: validasi tipe, isi, ukuran; nama acak; volume (A–B)	20%
Kamera/galeri, unggah multipart, pratinjau di Flutter (C)	20%
Foto di Filament (D)	5%
Suite pytest lengkap dan lulus (E)	20%
Tugas: laporan bulanan + ekspor CSV + cakupan $\geq 85\%$	20%
Tugas: ekspor Filament sesuai filter + kartu bulanan Flutter	15%

8. Pertanyaan Refleksi

1. Mengapa server tidak memakai nama berkas yang dikirim klien?
2. Mengapa memeriksa `Content-Type` saja tidak cukup?
3. Apa kelebihan menyimpan path relatif dibanding URL lengkap di database?
4. Mengapa foto diunggah setelah kegiatan tersimpan? Apa yang terjadi bila unggahan gagal?
5. Bagian mana dari kode Anda yang belum tercakup test, dan mengapa?

9. Troubleshooting Umum

Masalah	Solusi
413 dari server padahal foto kecil di galeri	<code>image_picker</code> tanpa <code>maxWidth</code> dapat mengirim foto beresolusi penuh. Atur <code>maxWidth/imageQuality</code>
<code>PermissionError</code> saat menulis <code>/data/uploads</code>	Volume dibuat dengan pemilik lain. Saat pengembangan: <code>docker compose down lalu docker volume rm tabungku_uploads</code>
Gambar tidak tampil di HP (<code>Image.network</code> galat)	URL masih memakai <code>localhost</code> . Pastikan <code>buktiUrl</code> memakai <code>apiUrl.value</code>
Gambar tidak tampil di Filament	<code>API_PUBLIC_URL</code> harus alamat yang dapat dibuka browser admin , bukan <code>http://api:8000</code>
iOS crash saat membuka kamera	Kunci <code>NSCameraUsageDescription</code> belum ada di <code>Info.plist</code>
<code>pytest: no such table</code>	Model tidak terimpor sebelum <code>create_all</code> . Impor <code>app.main</code> (yang mengimpor semua model) di <code>conftest.py</code>
pytest memakai MySQL sungguhan	<code>DATABASE_URL</code> harus di-set sebelum <code>from app...</code> . Perhatikan urutan impor di <code>conftest.py</code>
Peringatan <code>StarletteDeprecationWarning</code> ... <code>install httpx2</code>	Gunakan <code>httpx2</code> sebagai pengganti <code>httpx</code> di <code>requirements-dev.txt</code>

10. Referensi

- FastAPI, *Request Files*: fastapi.tiangolo.com/tutorial/request-files
- FastAPI, *Testing* dan *Testing Dependencies with Overrides*: fastapi.tiangolo.com/tutorial/testing
- OWASP File Upload Cheat Sheet: cheatsheetseries.owasp.org
- `image_picker`: pub.dev/packages/image_picker
- `pytest`: docs.pytest.org